Introduction to the Theory of Computation

Lecture 17. The Complexity Class NP

Neo (Yuan) Li

Nov 2026

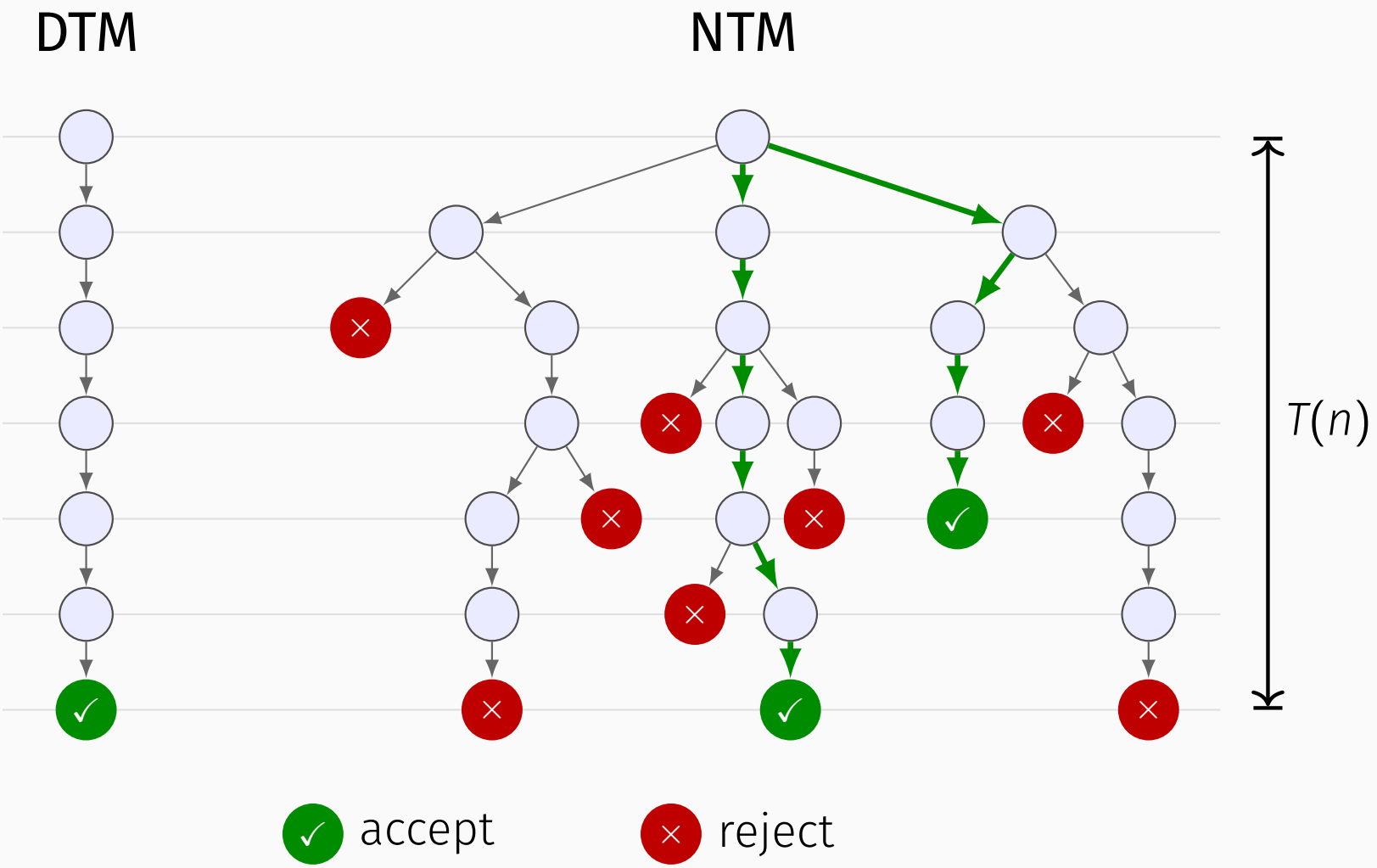
Fudan University

yuan_li@fudan.edu.cn

Motivation: A Different Kind of Machine

- So far: every TM is **deterministic** — one computation path.
- Today: at each step, the machine may **branch** into several next moves.
- This is **nondeterminism**, which defines NP.
- The biggest open question in TCS: is $P \neq NP$?

Deterministic vs. Nondeterministic Computation



Nondeterministic Turing Machine: Formal Definition

Definition 1 (Non-deterministic Turing machine)
An *NTM* is a 7-tuple $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$, where

1. Q is the set of states,
2. Σ is the input alphabet, not containing the blank symbol $\sqcup$,
3. Γ is the tape alphabet, where $\sqcup \in \Gamma$ and $\Sigma \subseteq \Gamma$,
4. $\delta : Q \times \Gamma \to \mathcal{P}(Q \times \Gamma \times \{L, R\})$ is the transition function,
5. $q_0 \in Q$ is the start state,
6. $q_{\text{accept}}, q_{\text{reject}} \in Q$ are the accept/reject states, $q_{\text{accept}} \neq q_{\text{reject}}$.

Configurations and Yields

Definition 2 (Configuration)
A **configuration** is a string uqv ($u, v \in \Gamma^*$, $q \in Q$):
tape $= uv$, state $= q$, head on the first symbol of v.

Definition 3 (Yields)
$C_1 \to C_2$ if M can go from C_1 to C_2 in one step, i.e., via some $(q', b, D) \in \delta(q, a)$.

What Does "M Accepts x" Really Mean?

Definition 4 (Acceptance)
NTM M *accepts* x if there exist configurations $C_0, C_1, \ldots, C_k$ with

1. $C_0 = q_0 x$ (start configuration),
2. $C_i \to C_{i+1}$ for all $0 \leq i < k$,
3. C_k is in state q_{accept}.

Definition 5
Let M be an NTM. The language accepted by M is $L(M) = \{ x \in \Sigma^* : M \text{ accepts } x \}$.

Running Time and Deciding a Language

Definition 6
NTM M *runs in time* $T(n)$ if for every input $x \in \{0,1\}^*$ and every sequence of nondeterministic choices, M halts within $T(|x|)$ steps.

Definition 7
NTM M *decides* a language $L \subseteq \{0,1\}^*$ if for every $x \in \{0,1\}^*$:

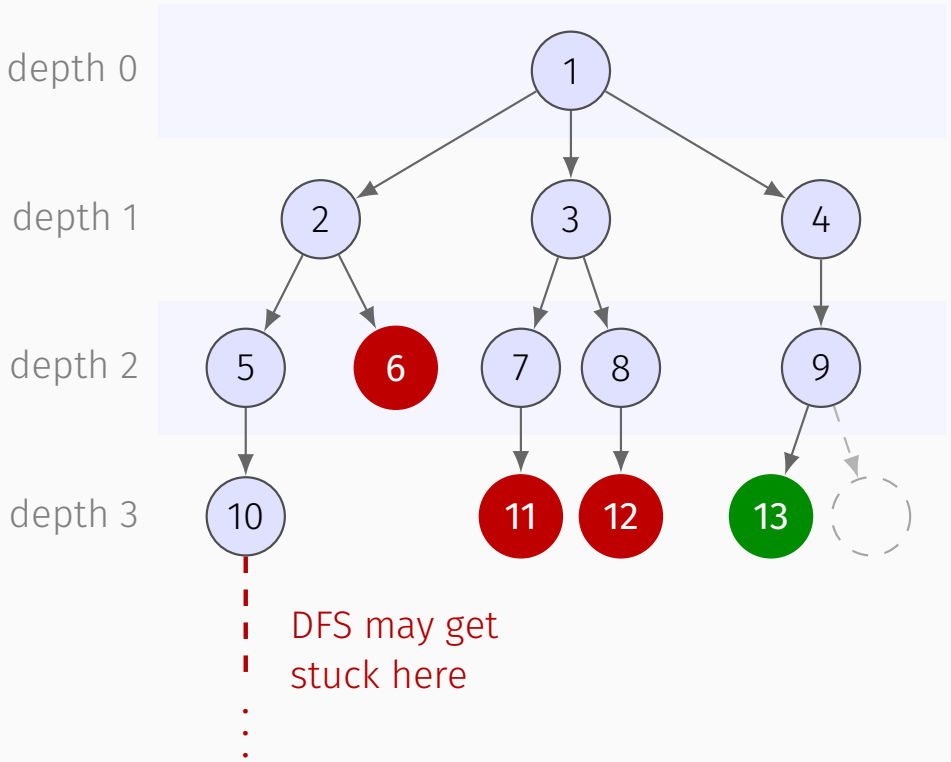
- if $x \in L$, then M accepts x;
- if $x \notin L$, then every computation path of M on x rejects.

Simulating an NTM Deterministically

Theorem 8

Every NTM N has an equivalent DTM D, i.e., $L(D) = L(N)$. Moreover, if N runs in time $T(n)$, then D runs in time $2^{O_N(T(n))}$.

Proof Sketch: Breadth-First Search



- D explores the computation tree of N *level by level*.
- Accept as soon as an accepting configuration is found.
- At most $b^{T(n)}$ configurations, where $b = \max |\delta(q, a)|$.

A Cleaner NTM: Binary-Choice Machines

Definition 9 (Binary-choice NTM)
A *binary-choice NTM* is an 8-tuple

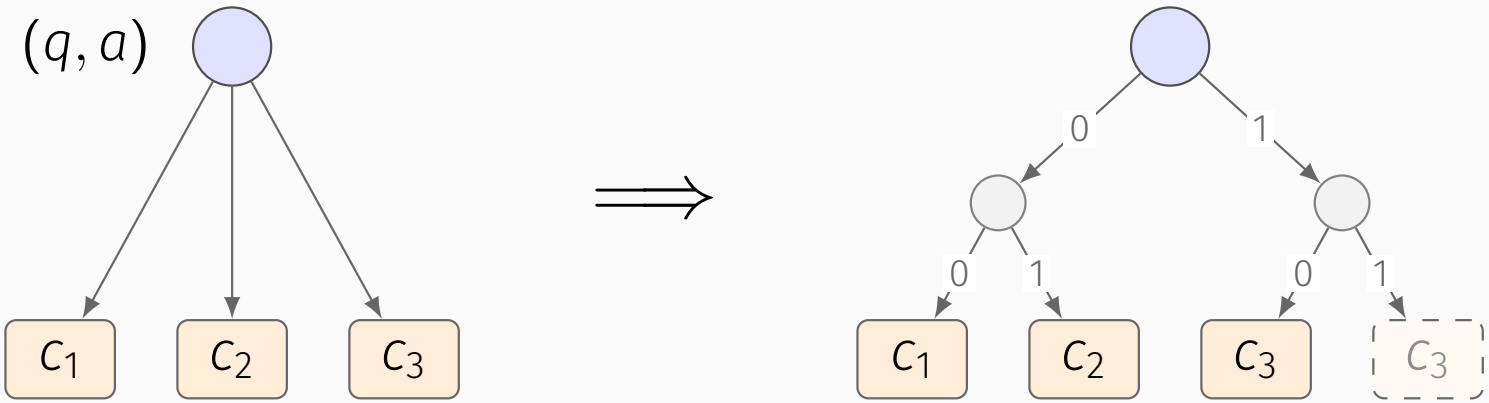
$$M = (Q, \Sigma, \Gamma, \delta_0, \delta_1, q_0, q_{\text{accept}}, q_{\text{reject}}),$$

where $\delta_0, \delta_1 : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$ are two *deterministic* transition functions. At each step, M nondeterministically applies δ_0 or δ_1.

From General NTMs to Binary-Choice NTMs

Lemma 10

If L can be decided by an NTM M in time $T(n)$, then L can be decided by a binary-choice NTM M' in time $O_M(T(n))$.



M: one step, $k = 3$

M': $d = \lceil \log_2 3 \rceil = 2$ bits

Proof of the Lemma

Let $K = \max_{q,a} |\delta(q,a)| \leq 2|Q||\Gamma|$ and $d = \lceil \log_2 K \rceil$. Fix an ordering $\delta(q,a) = \{c_1, \ldots, c_k\}$.

M' simulates one step of M (state q, reading a) as follows:

1. *Guess:* make d binary choices $b_1 \cdots b_d$, storing them in the finite control (each choice: move R, then a deterministic step back L).
2. *Execute:* apply c_j with $j = \min(b+1,\ k)$, where b is the number with binary representation $b_1 \cdots b_d$. If $k = 0$, enter q_{reject}.

Nondeterministic Time Classes

Definition 11

For $T : \mathbb{N} \to \mathbb{N}$, $\mathrm{NTIME}(T(n))$ is the class of languages $L \subseteq \{0,1\}^*$ decided by some multitape NTM in time $O(T(n))$.

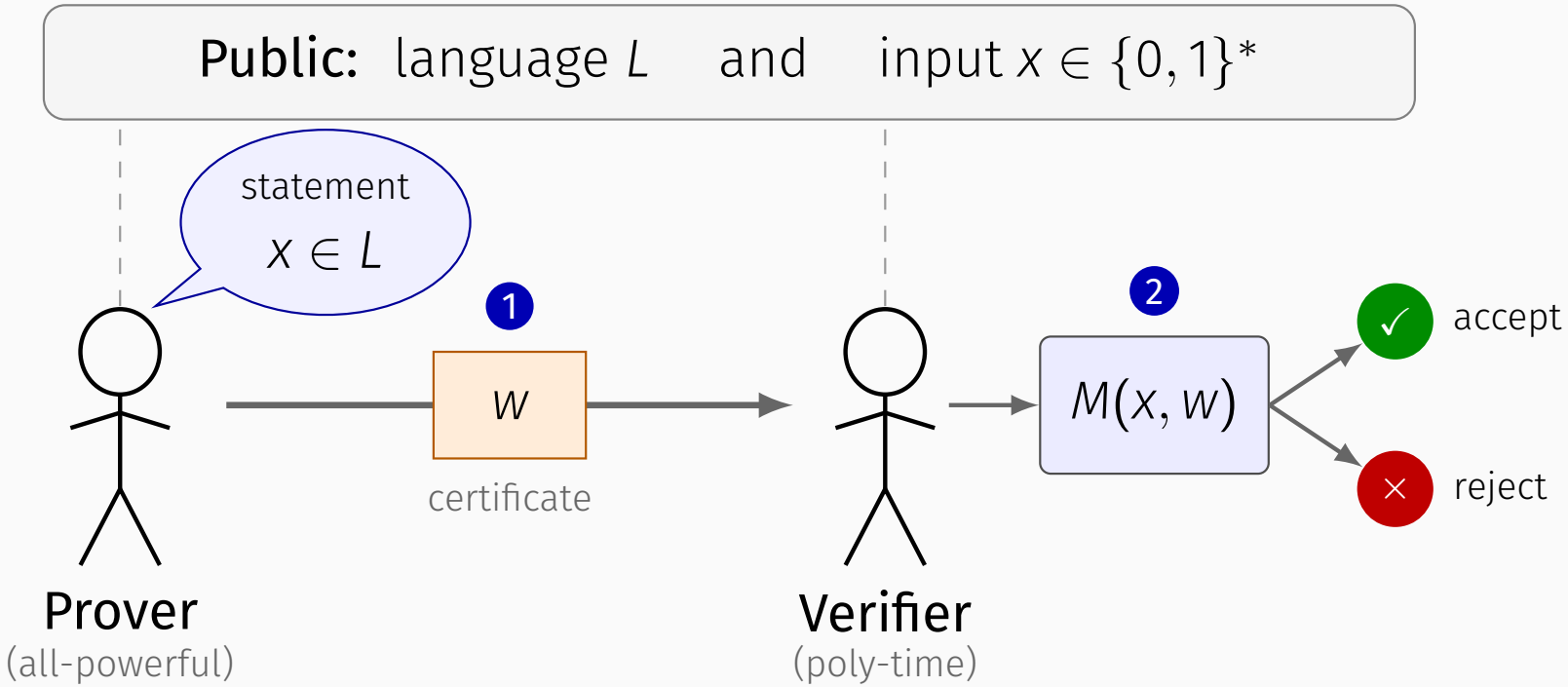
Definition 12

$$\mathrm{NP} = \bigcup_{c \geq 1} \mathrm{NTIME}(n^c).$$

Two Views

- P: **solvable** by a DTM in polynomial time.
- **NP** (view 1): **solvable** by an NTM in polynomial time.
- **NP** (view 2): **verifiable** by a DTM in polynomial time.

The Prover–Verifier View



1. Prover sends a **certificate** $w \in \{0,1\}^{p(|x|)}$.
2. Verifier runs a poly-time TM M and accepts iff $M(x, w) = 1$.

Completeness and Soundness

Completeness

$x \in L \implies \exists\, w :\ M(x, w) = 1$

an honest prover can convince

Soundness

$x \notin L \implies \forall\, w :\ M(x, w) = 0$

no prover can cheat

- Here w ranges over $\{0,1\}^{p(|x|)}$.
- Together: $x \in L \iff \exists\, w :\ M(x, w) = 1$.

Some Problems in NP

NP-complete

- SAT, 3SAT
- CLIQUE
- VERTEX COVER
- INDEPENDENT SET
- HAMILTONIAN CYCLE
- TSP
- SUBSET SUM
- GRAPH COLORING ($k \geq 3$)
- MAX CUT
- QUADEQ over $\mathrm{GF}(2)$

Believed in between

- GRAPH ISOMORPHISM
- INTEGER FACTORIZATION
- DISCRETE LOGARITHM

Not known to be in P, not believed to be NP-complete.

SAT: Boolean Satisfiability

- A **variable** x_i takes value 0 (FALSE) or 1 (TRUE).
- A **literal** is a variable x_i or its negation $\neg x_i$.
- A **clause** is an OR of literals, e.g. $(\neg x_1 \vee x_3)$.
- A **CNF formula** is an AND of clauses.

Definition 13

$\mathrm{SAT} = \{\varphi : \varphi \text{ is a satisfiable CNF formula}\}$.

SAT: Examples

Satisfiable

$$\varphi_1 = (x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3)$$

Take $x_1 = 1,\ x_2 = 0,\ x_3 = 1$:

$$(1 \vee 0) \wedge (0 \vee 1) \wedge (1 \vee 0) = 1. \checkmark$$

Unsatisfiable

$$\varphi_2 = (x_1 \vee x_2) \wedge (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2)$$

Each of the 4 assignments falsifies a clause:

$(0,0)$: 1st, $(0,1)$: 2nd,

$(1,0)$: 3rd, $(1,1)$: 4th.

3SAT

Definition 14

$3\mathrm{SAT}$ is the set of satisfiable CNF formulas in which every clause has exactly 3 literals.

Example

$$\varphi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_4) \wedge (x_2 \vee \neg x_3 \vee \neg x_4)$$

Take $x_1 = 1,\ x_2 = 0,\ x_3 = 0,\ x_4 = 1$:

$$(1 \vee 1 \vee 0) \wedge (0 \vee 0 \vee 1) \wedge (0 \vee 1 \vee 0) = 1. \checkmark$$

Certificate: a satisfying assignment, checkable in linear time.

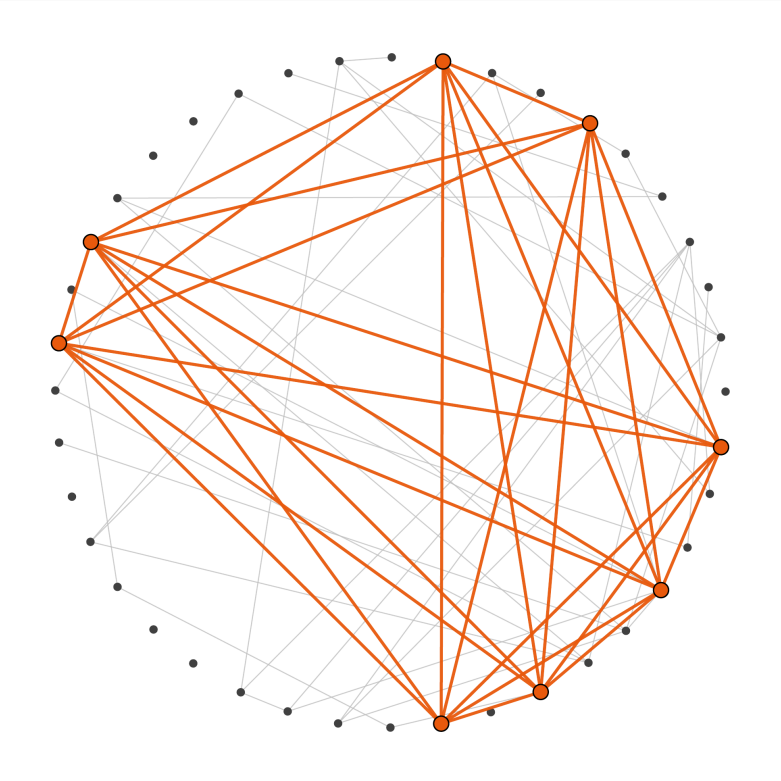
In-Class Exercise: Satisfiable or Not?

Is each of the following formulas satisfiable? Answer **Yes** or **No**.

1. $(x_1 \vee x_2) \wedge (\neg x_1) \wedge (\neg x_2)$
2. $(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_2) \wedge (x_1 \vee x_2)$
3. $(x_1) \wedge (\neg x_1 \vee x_2) \wedge (\neg x_2 \vee x_3) \wedge (\neg x_3)$
4. $(x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2) \wedge (\neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_3)$
5. $(x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_3)$

CLIQUE

$\text{CLIQUE} = \{\langle G, k\rangle : G \text{ has } k \text{ pairwise adjacent vertices}\}.$

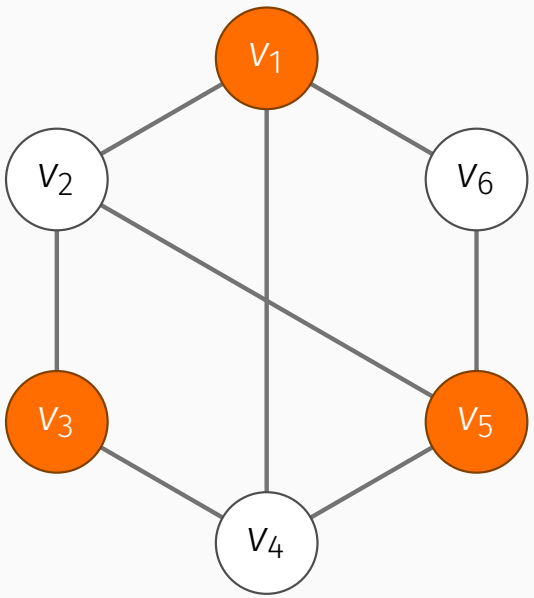


A K_8 (orange) hidden in a graph on 40 vertices.

VERTEX COVER

Definition 15
A *vertex cover* of $G = (V, E)$ is a set $S \subseteq V$ such that every edge in E has at least one endpoint in S.

$$\mathrm{VC} = \{\langle G, k\rangle : G \text{ has a vertex cover of size } k\}.$$



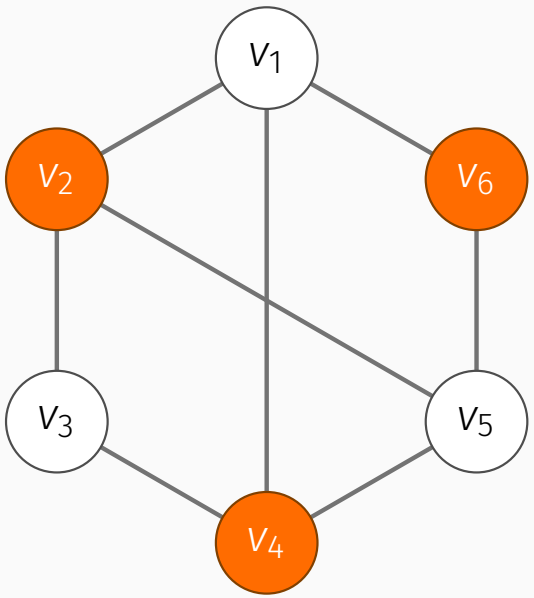
$S = \{v_1, v_3, v_5\}$ is a vertex cover of size 3: each of the 8 edges has an orange endpoint.

Certificate: S. Check every edge once.

INDEPENDENT SET

Definition 16
An *independent set* of $G = (V, E)$ is a set $S \subseteq V$ such that no edge has both endpoints in S.

$$\text{INDSET} = \{\langle G, k\rangle : G \text{ has an independent set of size } k\}.$$



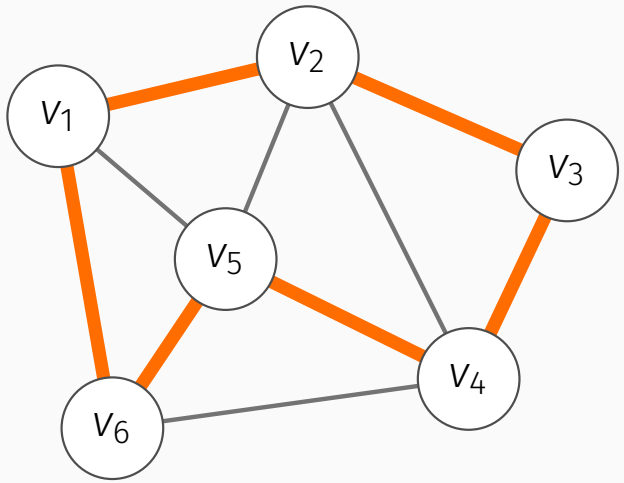
$S = \{v_2, v_4, v_6\}$ (orange) is an independent set of size 3: no two of them are adjacent.

Certificate: S. Check every pair in S is a non-edge.

HAMILTONIAN CYCLE

Definition 17
A *Hamiltonian cycle* of G is a cycle that visits every vertex exactly once.

$$\text{HAMCYCLE} = \{\langle G \rangle : G \text{ has a Hamiltonian cycle}\}.$$



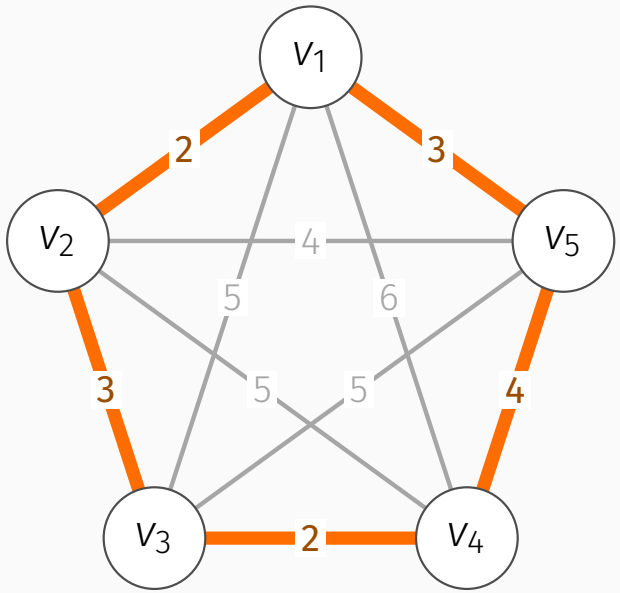
$v_1 \to v_2 \to v_3 \to v_4 \to v_5 \to v_6 \to v_1$ (orange) is a Hamiltonian cycle.

Certificate: the cyclic order. Check each consecutive pair is an edge.

TRAVELING SALESMAN PROBLEM

Definition 18
Let G be a complete graph with edge weights w. A *tour* is a Hamiltonian cycle of G; its weight is the sum of its edge weights.

$$\mathrm{TSP} = \{\langle G, w, B\rangle : G \text{ has a tour of weight} \leq B\}.$$



With $B = 14$: the tour
$v_1 \to v_2 \to v_3 \to v_4 \to v_5 \to v_1$
(orange) has weight
$2 + 3 + 2 + 4 + 3 = 14 \leq B$.

Certificate: the tour. Check it is a Hamiltonian cycle and add up the weights.

SUBSET SUM

Definition 19
Given a multiset S of integers and a target t:

$$\text{SUBSET-SUM} = \left\{ \langle S, t \rangle : \exists\, S' \subseteq S,\ \sum_{s \in S'} s = t \right\}.$$

$t = 15$: **yes**



$2 + 3 + 10 = 15$. ✓

$t = 22$: **no**

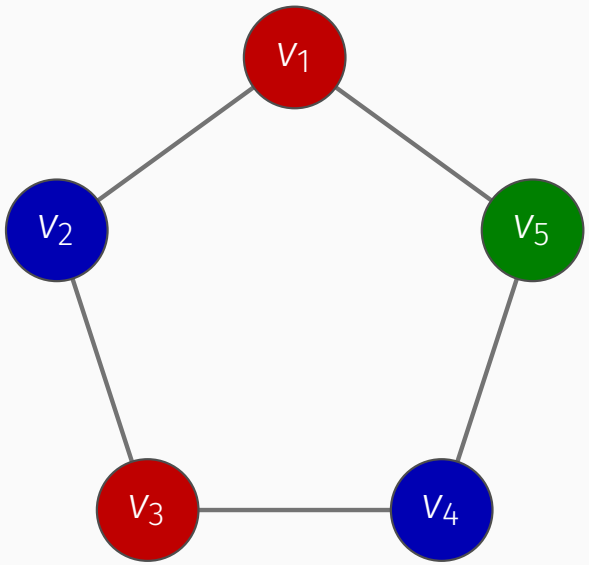


The total is 29, so a subset summing to 22 would leave the rest summing to 7. But no subset sums to 7.

GRAPH COLORING ($k \geq 3$)

Definition 20
A *proper k-coloring* of G assigns one of k colors to each vertex so that the two endpoints of every edge get different colors.

$$\text{COLOR} = \{\langle G, k\rangle : G \text{ has a proper } k\text{-coloring}\}.$$



C_5 with $k = 3$: every edge joins two different colors.

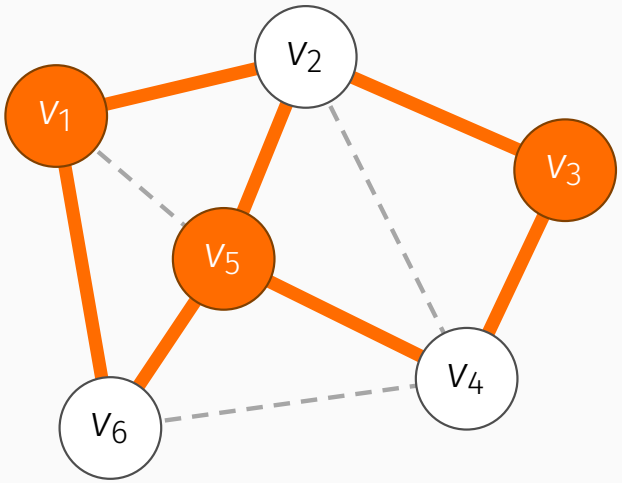
$k = 2$ is impossible: two colors must alternate around the cycle, but 5 is odd.

Certificate: the coloring. Check every edge once.

MAX CUT

Definition 21
A *cut* $(S, V \setminus S)$ splits the vertices into two sides; its size is the number of edges crossing between the sides.

$$\mathrm{MAXCUT} = \{\langle G, k\rangle : G \text{ has a cut of size} \geq k\}.$$



$S = \{v_1, v_3, v_5\}$ (orange) vs. the rest: 7 of the 10 edges cross (thick), so $\langle G, 7\rangle \in \mathrm{MAXCUT}$.

Certificate: S. Count the crossing edges.

QUADRATIC EQUATIONS over $\mathrm{GF}(2)$

Definition 22
A *quadratic system* is a set of m equations of degree ≤ 2 over $\mathrm{GF}(2)$ in variables $x_1, \ldots, x_n$.

$$\mathrm{QUADEQ} = \{\langle E \rangle : E \text{ is a quadratic system with a solution}\}.$$

(Arithmetic is mod 2: $+$ is XOR, $\cdot$ is AND.)

Example: $n = m = 3$

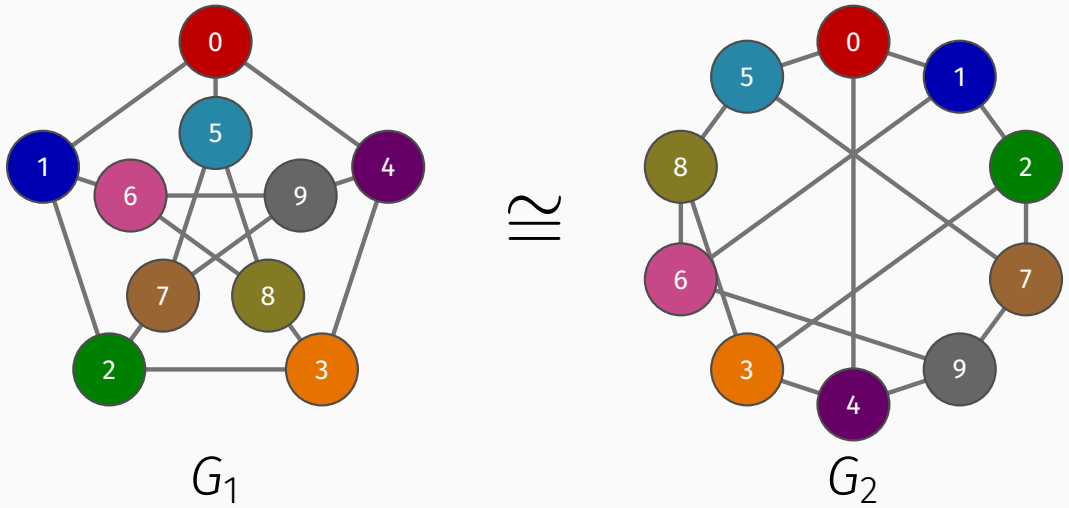
$$\begin{aligned} x_1x_2 + x_3 &= 1 \\ x_1 + x_2x_3 &= 1 \\ x_1x_3 + x_2 &= 1 \end{aligned} \qquad x = (1,1,0): \qquad \begin{aligned} 1+0 &= 1 \checkmark \\ 1+0 &= 1 \checkmark \\ 0+1 &= 1 \checkmark \end{aligned}$$

Certificate: the assignment x. Evaluate each equation.

GRAPH ISOMORPHISM

Definition 23
$G_1 \cong G_2$ if there is a bijection $\pi : V_1 \to V_2$ with
$\{u, v\} \in E_1 \iff \{\pi(u), \pi(v)\} \in E_2$.

$$\mathrm{GI} = \{\langle G_1, G_2 \rangle : G_1 \cong G_2\}.$$



G_1 G_2

Certificate: π (here, each label maps to itself). Check every edge.

Breakthrough (Babai, 2015): first *quasipolynomial*-time algorithm for GI, $n^{O((\log n)^c)}$.

INTEGER FACTORIZATION

Definition 24
Given integers N, L, U in binary, with $1 < L \leq U < N$:

$$\mathrm{FACTOR} = \{\langle N, L, U\rangle : N \text{ has a divisor } d \text{ with } L \leq d \leq U\}.$$

Example: $N = 391,\ L = 10,\ U = 20$
$d = 17$: $\quad 10 \leq 17 \leq 20$ ✓ $\qquad 391 = 17 \times 23$ ✓

Certificate: d, at most $n = O(\log N)$ bits. Check $L \leq d \leq U$ and $N \bmod d = 0$.

Classically hard (RSA relies on it), but easy on a quantum computer (Shor, 1994).

How Fast Can We Factor?

Input: an n-bit integer N ($n \approx \log_2 N$).

Algorithm	Time	
Trial division	$2^{n/2}$	exponential
Quadratic sieve (1980s)	$2^{\tilde{O}(n^{1/2})}$	subexponential
Number field sieve (1990s)	$2^{\tilde{O}(n^{1/3})}$	best known classical
Shor (1994), *quantum*	$\mathrm{poly}(n)$	polynomial

Records. Classical: RSA-250 (829 bits), 2020, $\approx$ 2700 core-years. Quantum estimate: RSA-2048 with $< 10^6$ noisy qubits in $<$ 1 week (Gidney, 2025).

DISCRETE LOGARITHM

Definition 25
Given a prime p, a generator g of $\mathbb{Z}_p^*$, an element $h \in \mathbb{Z}_p^*$, and bounds $L \leq U$:

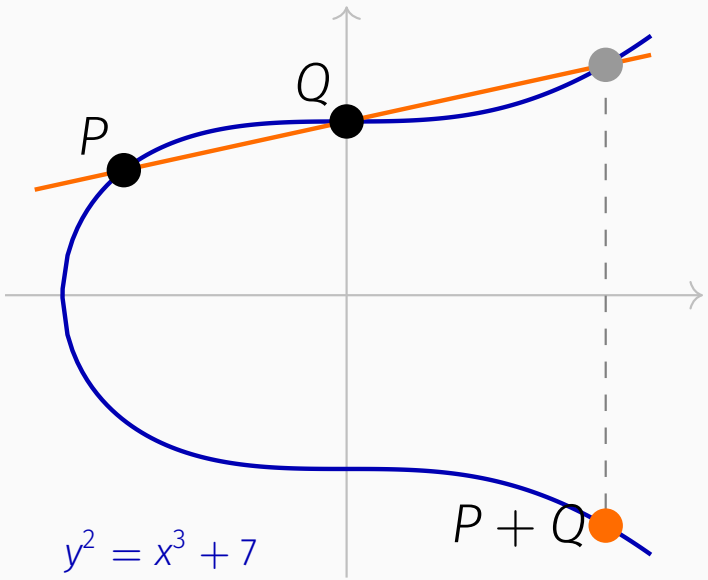
$$\mathrm{DLOG} = \{\langle p, g, h, L, U\rangle : \exists x \in [L, U],\ g^x \equiv h \pmod{p}\}.$$

Example: $p = 23,\ g = 5,\ h = 8,\ [L, U] = [1, 10]$
$x = 6$: $\quad 5^6 = (5^2)^3 \equiv 2^3 = 8 \pmod{23}$ ✓

Certificate: x. Check $L \leq x \leq U$ and compute $g^x \bmod p$ by repeated squaring.

Classically hard (Diffie–Hellman relies on it), but easy on a quantum computer (Shor, 1994).

Frontier: The Group Behind Bitcoin



(drawn over $\mathbb{R}$ for intuition)

secp256k1: points (x, y) with $y^2 = x^3 + 7$ over $\mathbb{F}_p$, $p = 2^{256} - 2^{32} - 977$, plus a point $\mathcal{O}$ at infinity.

Group law: draw the line through P, Q, take the third point, reflect. The group has prime order $n \approx 2^{256}$ and a fixed generator G.

Keys: private key k, public key $K = kG = \underbrace{G + \cdots + G}_{k}$.

ECDLP: given G and $K = kG$, find k

Easy: computing kG takes ≤ 512 group operations.

Hard: the best known classical attack takes $\sim 2^{128}$ steps.

How Fast Can We Compute Discrete Logs?

Input size n bits; group order $q \approx 2^n$.

Group	Best known algorithm	Time
$\mathrm{GF}(2^n)$, small characteristic	quasi-polynomial (2014)	$n^{O(\log n)}$
$\mathbb{Z}_p^*$	number field sieve	$2^{\tilde{O}(n^{1/3})}$
Elliptic curves (secp256k1)	Pollard rho, generic	$2^{n/2}$
Any of the above, *quantum*	Shor (1994)	$\mathrm{poly}(n)$

Why Bitcoin uses elliptic curves

No shortcut beyond generic algorithms is known, and generic algorithms need $\Omega(\sqrt{q})$ steps (Shoup, 1997). So secp256k1 ($n = 256$) gives $\sim 2^{128}$ security.

Records. Classical: $\mathbb{Z}_p^*$ with 795-bit p (2019); elliptic curve with 114-bit group (2017).

Quantum estimate: a secp256k1 private key with $< 5 \times 10^5$ noisy qubits in $\sim$ minutes (Babbush et al., 2026).

Definition 2 of NP: The Verifier Definition

Theorem 26

$L \in \mathrm{NP}$ if and only if there exist a poly-time TM M and a polynomial p such that for every $x \in \{0,1\}^*$:

- *Completeness: if $x \in L$, then $\exists\, w \in \{0,1\}^{p(|x|)}$ with $M(x,w) = 1$;*
- *Soundness: if $x \notin L$, then $\forall\, w \in \{0,1\}^{p(|x|)}$, $M(x,w) = 0$.*

Proof of Equivalence: NTM $\Rightarrow$ Verifier

Let L be decided by a binary-choice NTM N in time $p(n) = c_2 n^{c_1}$.

- **Certificate:** $w \in \{0,1\}^{p(|x|)}$, where bit i says whether N applies δ_0 or δ_1 at step i.
- **Verifier $M(x, w)$:** run N on x, resolving the choices by w; accept iff it reaches q_{accept}.
- **Correctness:** $x \in L \iff$ some path accepts $\iff$ some w is accepted.

Proof of Equivalence: Verifier $\Rightarrow$ NTM

Let poly-time M and polynomial p be a verifier for L.

- **NTM $N(x)$:** nondeterministically guess $w \in \{0,1\}^{p(|x|)}$, then run $M(x, w)$; accept iff M accepts.
- **Time:** $p(|x|)$ steps to guess, plus the running time of M, which is polynomial.
- **Correctness:** $x \in L \iff$ some w is accepted $\iff$ some path of N accepts.

Definition of NP, Two Ways

- **Nondeterminisitc TM:** $\mathrm{NP} = \bigcup_{c \geq 1} \mathrm{NTIME}(n^c)$.
- **Verifier:** $x \in L$ iff some poly-size certificate w makes a poly-time M accept.
- **Equivalent:** guessing $w \leftrightarrow$ choice bits of a binary-choice NTM.
- **Intuition:** easy to check, possibly hard to solve.

Summary: P and NP

- P: languages **solvable** in polynomial time.
- NP: languages **verifiable** in polynomial time, given a certificate w.

Every example today (SAT, CLIQUE, VERTEX COVER, ...) has the same shape: **checking** an answer is easy. Is **finding** one just as easy?

Beyond One-Round Verification

- **Interactive proofs:** many rounds of back-and-forth. $\mathrm{IP} = \mathrm{PSPACE}$.
- **PCPs:** verifier reads only $O(1)$ bits of w, using $O(\log n)$ random bits.
  $\mathrm{NP} = \mathrm{PCP}[O(\log n), O(1)]$.
- **Zero-knowledge proofs:** prove $(\exists w)(y = f(x, w))$ without revealing w.